TP “Motor de búsqueda de funciones similares vía AST”

Lucas Aimetta, Francisco Ambort

Decisiones de diseño.

A la hora de diseñar el sistema, siguiendo las recomendaciones del profesor, optamos por dividir en módulos independientes cada funcionalidad, estructurado en un pipeline de 4 fases:

1. Extracción:

   Busca y lee todos los archivos .py del proyecto, para aislar cada función, transformando el código en una estructura de árbol.

2. Normalización:

   Limpia y normaliza el árbol de cada función, borrando nombres particulares y valores fijos, siendo estos reemplazados por valores genéricos, haciendo que dos códigos que hacen los mismo se vean idénticos.

3. Serialización:

   Aplana la estructura del árbol, recorriendo de arriba abajo para convertirla en una lista ordenada y secuencial.

4. Comparación:

   Mide matemáticamente cuánto se parecen dos listas, comparando fragmentos para devolver un porcentaje de similitud e indicar si son duplicados.

Además contamos con un quinto módulo CLI, el cuál se encarga de orquestar el resto de módulos y de interactuar con el usuario, solicitando una serie de parámetros y dandole como respuesta las X funciones más similares dentro del directorio pasado.

Los parámetros que el usuario ingresa son:

- **dir**: Es la ruta al directorio de archivos Python a analizar.
- **top**: Es la cantidad de resultados similares a mostrar. Por defecto es 5.
- **umbral**: Es la frontera para decidir si un resultado es una copia de la función a analizar. Por defecto es 0.8.
- **función/código**: Este parámetro permite ingresar el nombre de la función en formato 'archivo.py:nombre_funcion' o el código de una función Python a analizar.

Este último parámetro presenta dos opciones las cuáles son mutuamente excluyentes, cómo procesara el sistema la función a analizar dependerá de cuál se elige, si optamos por pasar el nombre de una función, el sistema busca la función dentro de los archivos del directorio indicado, por el otro lado, si opta por pasar código, parsea directamente el texto que el usuario escribió en la terminal.

A partir de aquí, en ambos casos el sistema hace exactamente lo mismo: obtiene el árbol sintáctico (AST) de la función objetivo, lo normaliza reemplazando los nombres de variables y literales por tokens genéricos, y lo convierte en una lista de tokens para luego compararlos.

Métrica elegida.

Para cuantificar la similitud estructural se implementó el coeficiente de Jaccard sobre n-gramas generados a partir del AST normalizado , el cual devuelve un valor entre 0.0 y 1.0

Se seleccionó este método porque opera sobre conjuntos de subestructuras (n-gramas), esto logra una gran tolerancia al reordenamiento de sentencias independientes dentro de una función

Cuando dos líneas de código intercambian su lugar, la gran mayoría de los n-gramas que componen el cuerpo de cada instrucción se conservan idénticos dentro del conjunto. De este modo, la intersección entre ambos conjuntos sigue siendo casi igual, permitiendo que funciones duplicadas donde únicamente se alteró el orden de ejecución mantengan un puntaje de similitud elevado.

A la hora de decidir que N elegimos para hacer nuestros n-gramas, optamos por $n = 3$ porque nos garantiza un equilibrio entre precisión ya que valores con $n \leq 2$ capturan solo pares de tokens genéricos inflando la similitud en códigos no relacionados, y entre tolerancia debido a que valores con $n \geq 5$ provocan sobreajuste, haciendo que una sola línea agregada invalide múltiples secuencias contiguas y cause falsos negativos.

Limitaciones conocidas

Si bien nuestro motor funciona correctamente a la hora de encontrar funciones similares en su estructura, es decir, de forma sintáctica, presenta inconsistencias a la hora de encontrar funciones que logran el mismo resultado pero cambiando su estructura, en otras palabras, no analiza semánticamente.

Un claro ejemplo de esto es a la hora de analizar un algoritmo que hace una multiplicación utilizando el símbolo * y otro que la hace con un bucle y el símbolo +.

Ambas funciones llegan al mismo resultado, sin embargo, como su estructura interna cambia, nuestro motor de búsqueda no los considera iguales.